

高阶函数：函数作为参数

[[theory_11_closure_decorator]] · [[课堂老师程序-函数专题对照]]

核心思想：把"做什么"和"什么时候做"分开

一个最简单的例子——为什么要把函数当参数传？

```
def run(func, *args, **kwargs):  
    print("开始执行")  
    result = func(*args, **kwargs)  
    print("执行结束")  
    return result
```

`run` 的价值不在于它能调用函数，而在于它能在调用函数前后做通用的事情。

一、对比：不用高阶函数 vs 用高阶函数

✗ 不用高阶函数：每个函数都要改

```
def say_hello(name):  
    print("开始执行")          # 重复代码  
    result = f"你好, {name}!"  
    print("执行结束")          # 重复代码  
    return result  
  
def add(a, b):  
    print("开始执行")          # 重复代码  
    result = a + b  
    print("执行结束")          # 重复代码  
    return result  
  
def multiply(a, b):
```

```
print("开始执行")          # 重复代码
result = a * b
print("执行结束")          # 重复代码
return result
```

问题： 100 个函数就要改 100 遍。

✅ 用高阶函数：通用逻辑只写一次

```
def run(func, *args, **kwargs):
    print("开始执行")
    result = func(*args, **kwargs)
    print("执行结束")
    return result
```

原函数保持干净

```
def say_hello(name):
    return f"你好, {name}!"
```

```
def add(a, b):
    return a + b
```

调用时传函数名

```
run(say_hello, "李四")
run(add, 3, 4)
```

二、和装饰器的关系

run 函数（手动调用）

```
def run(func, *args, **kwargs):
    print("开始执行")
    result = func(*args, **kwargs)
    print("执行结束")
    return result
```

装饰器（自动调用）——本质一样

```
def logger(func):
```

```
def wrapper(*args, **kwargs):  
    print("开始执行")  
    result = func(*args, **kwargs)  
    print("执行结束")  
    return result  
  
return wrapper
```

区别： `run` 是手动传函数调用，装饰器是 `@logger` 标记后自动包装。

三、三个实战价值

1. 动态选择执行哪个函数（替代 if-elif 链）

```
def add(a, b): return a + b  
def sub(a, b): return a - b  
def mul(a, b): return a * b  
  
# 映射表代替 if-elif  
ops = {"+": add, "-": sub, "*": mul}  
  
op = input("操作符: ")  
a, b = int(input()), int(input())  
  
if op in ops:  
    print(ops[op](a, b))    # 动态选择函数
```

2. 批量执行多个函数

```
def square(x): return x ** 2  
def cube(x): return x ** 3  
def double(x): return x * 2  
  
funcs = [square, cube, double]  
for f in funcs:  
    print(f(5))    # 25, 125, 10
```

3. 在调用前后做通用操作（日志/计时/权限）

```
import time

def timer(func, *args, **kwargs):
    start = time.time()
    result = func(*args, **kwargs)
    elapsed = time.time() - start
    print(f'{func.__name__} 耗时: {elapsed:.4f}秒')
    return result
```

四、什么时候用

场景	直接调用	函数作为参数
只调一个特定函数	✅ 推荐	❌ 多余
需要在调用前后做通用操作	❌ 重复代码	✅ 一次编写到处用
需要动态选择执行哪个函数	❌ if-elif 冗长	✅ 映射表
需要批量执行多个函数	❌ 一个个写	✅ 简洁

五、传入函数和返回函数是对称的



两者合在一起就是 Python 装饰器的完整实现。